



Departamento de electrónica e informática

FUNDAMENTOS DE PROGRAMACIÓN (FDP)

Proyecto práctico basado en programación estructurada

Nombre del Proyecto:
Pokémon: Héroes de Kanto

Estudiantes:

Guerra Aquino, Juan José, 00079926

Meléndez Rivas, Edgard Daniel, 00071126

Avalos Hernández, Gabriel Eduardo, 00264126

Pérez Interiano, Diego Xavier, 00242126

Fecha de entrega: 8/07/2026

Descripción general del juego o sistema

Es un videojuego de simulación de batalla Pokémon PvE (Jugador contra el entorno) desarrollado en C++ bajo el paradigma de programación estructurada. El proyecto implementa un bucle de juego basados en ciclos while y for para ir controlando los turnos, estructura switch e if else para el manejo de decisiones y menú, también arreglos donde se van guardando los datos del jugador.

Además, para el desarrollo de este juego se ha hecho uso de la programación modular, dividiendo la lógica del sistema en archivos de cabecera (.h) y de implementación (.cpp). Esta estructura permite mantener un código limpio, ordenado que evita la estructura espagueti tradicional y que lo hace escalable para futuras mejoras

Objetivo del proyecto

Desarrollar un video juego funcional en consola haciendo uso practico de estructuras básicas de la programación estructurada en C++, permitiendo comprender a fondo la lógica de control de flujo, a la organización de datos en la memoria y manipulación de archivos para resolver problemas reales de desarrollo.

Reglas o funcionamiento

1. **Selección de personaje:** El usuario puede elegir entre 3 Pokémon diferentes disponible en el juego para enfrentarse a un Pokemon generado por el sistema.
2. **Set de movimientos:** Cada Pokémon que conforma el juego cuenta con un total de 4 ataques únicos y diferenciados.
3. **Sistema de combate 1v1 por turno:** El jugador elegi uno de los diferentes ataques con los que cuenta el pokemon, reduce vida al contricante y si este sobrevive devuelve un ataque con daño aleatorio generado.
4. **Condición de victoria/derrota:** El combate finaliza cuando alguno de los puntos de HP (vida) de uno de los Pokémon llegue a cero.



Opciones del menú

1. **Cargar partida:** Al iniciar, el juego pregunta si desea cargar una partida ya existente. Si no hay o se rechaza continua a la selección del Pokémon.
2. **Menú principal:** Este es el bucle central que permite al usuario elegir entre:
 - A. Luchar (inicia la función de combate).
 - B. Guardar partida (sobrescribe los datos actuales en el archivo de texto).
 - C. Ver estadísticas de tu pokemon (muestra vida, experiencia, fuerza y nivel actual).
 - D. Ver movimientos de tu pokemon (muestra el set de 4 ataques).
 - E. Salir (cierra el bucle y termina el programa).

Explicación del uso de los temas vistos en clase

1. **Tipos de datos:** El proyecto utiliza una estructura de datos tipada para gestionar la información del juego de manera eficiente:
 - **String:** Se utiliza para almacenar cadenas de texto descriptivas, incluyendo los nombres de los Pokémon, las descripciones detalladas de los ataques y el texto narrativo.
 - **Arreglos tipo string:** Se implementa arreglos unidireccionales (ej. Movimientos_squirtle[4]) para organizar el set de 4 movimientos de cada Pokémon. Cada posición del arreglo contiene el nombre del ataque, su tipo (físico, especial, etc.) y su efecto mecánico (reducción de HP o estados).
 - **Booleanos:** Se utiliza para el proceso de carga de partidas, ya que si el archivo existe retorna true, pero si no existe retorna false.
 - **Char:** Se utiliza para recibir un carácter, este tipo de dato se ha utilizado para evaluar la opcion del menu que ha elegido el usuario.
 - **Int (Enteros):** Se emplea para todas las variables que requieren gestión numérica de estados y lógica de control. Esto incluye:

- **Estadísticas de combate:** vida de ambos jugadores y daño de cada ataque.
 - **Control de flujo:** Aplicado a los menús y las distintas selecciones que debe hacer el usuario.
 - **Gestión de archivos:** Se emplea en la guardado de datos y proceso de lectura/ escritura.
2. **Operadores:** Uso de operadores aritméticos (-, +) para restar la vida tras un ataque; operadores relacionales (>, <=) para verificar si la vida llego a cero, y operadores logicos (&&, ||) para validar condiciones de salida o de selección del juego.
 3. **Condicionales:** Utilizados para validar que el usuario ingrese opciones correctas, por ejemplo, verificar qué Pokémon elegio el usuario o qué tipo de ataque escogió.
 4. **Estructura switch:** Utilizado para evaluar la letra que ingresa el usuario (A, B, C,D mayúscula o minúscula) y ejecutar el ataque seleccionado.
 5. **Ciclo while:** Funciona como el bucle principal del juego que mantiene la batalla activa de forma indefinida mientras ambos Pokémon tengan vida (HP >0).
 6. **Ciclo for:** Utilizado para recorrer los arreglos de datos de forma secuencial, por ejemplo, para imprimir en pantalla la lista de 4 ataques disponibles del Pokémon
 7. **Arreglo/arrays:** Se utilizaron arreglos para el manejo estructurado de datos relacionados, permitiendo agrupar elementos bajo un mismo identificados para facilitar su acceso y recorrido.
 - **Arreglos de tipo string:** Implementación de arreglos unidimensionales de tamaño fijo, tales como los movimientos de cada Pokémon. Estos arreglos actúan como una base de datos local que almacena los 4 ataques únicos de cada Pokémon.
 - **Structs:** Se utilizaron para guardar y agrupar distintas características de un personaje (nombre, vida, experiencia, nivel, fuerza) en un solo tipo de dato manejable.
 - **Ventaja en el flujo de juego:** El uso de estos arreglos nos permite acceder de forma rapida y eficiente a la información específica de cada

Pokémon mediante indies (del 0 al 3), evitando la redundancia de código y facilitando la implementación de los menús de selección y validaciones de combate dentro del ciclo

- Flujograma digital

[Flujograma-PokeCoders](#)

Explicación del manejo de archivos

El proyecto hace uso de librerías de manejo de archivos en C++ para dos funciones esenciales que garantizan la persistencia de la información:

- **Lectura de datos (ifstream):** Permite al programa cargar externamente desde datos_partida.txt las estadísticas guardadas del jugador (nombre, vida, nivel, etc.) para reanudar el juego.
 - **Escritura de datos (ofstream):** Se utiliza cuando el jugador selecciona la opción de “Guardar” en el menú, sobrescribiendo las estadísticas actuales de la partida en él (.txt) que se ha generado.
- Referencias o recursos de apoyo utilizados, si aplica

Parte estética del proyecto y de guardado de datos fue apoyada con Inteligencia Artificial (ChatGPT)